

Predict Once, Query Many: World Models that Forecast Conditional Trajectory Abstractions

Anonymous CVPR submission

Paper ID *****

Abstract

001 A world model predicts how the environment will respond
002 to an agent's actions, and a planner uses these predictions
003 to choose among candidate actions. Latent world models
004 such as DINO-WM predict the visual features of every future
005 frame, and plan by comparing the last predicted frame with
006 a goal image in feature space. When a robot must choose
007 among a few candidate action chunks, much of this predic-
008 tion restates what the history already shows, and feature
009 distance to a goal image can be a poor measure of task
010 progress. We ask whether a world model can instead predict
011 a compact abstraction of an action's entire future that keeps
012 what the decision needs and omits what the history already
013 reveals. We propose Conditional Trajectory Abstraction
014 (CTA), a joint-embedding world model whose prediction tar-
015 get is learned. During training, a target encoder compresses
016 the actual future of an action chunk into 16 discrete tokens,
017 conditioned on the observed history. A goal-conditioned
018 reader ranks sibling futures by task progress from the history
019 and this code alone, and a feature decoder reconstructs the
020 future's frozen image features, so that the code describes
021 the future instead of merely scoring the training goals. The
022 world model predicts the code from the history and a can-
023 didate chunk in a single forward pass. Because the code
024 does not depend on the goal, each candidate is predicted
025 once and can then be evaluated by the reader for any goal.
026 Selecting among the proposals of a frozen diffusion policy on
027 PushT, CTA [main result]. An attribution ladder separates
028 the ranking quality lost by compressing the future from that
029 lost by predicting it.

030 1. Introduction

031 A world model predicts how the environment evolves in
032 response to an agent's actions [17, 18]. Prediction alone
033 does not choose an action: a planner evaluates the predicted
034 outcomes of candidate actions against a goal, for example
035 with a cost or a learned reward. In visual control, a prominent

line of work learns the world model in the feature space of a frozen pretrained encoder. DINO-WM [38] and V-JEPA 2-AC [3] predict the patch features of every future frame, and plan by minimizing the distance between the features of the last predicted frame and those of a goal image. Predicting features avoids generating pixels, and because the goal enters only through the cost, one world model serves any goal image.

When a robot must choose among candidate action chunks, however, a per-step latent target spends computation where the decision does not need it and leaves open what the decision does need. Each candidate requires predicting a full grid of feature tokens for every future step, 2,048 tokens for an 8-step chunk in our PushT setup, even though the scene layout, the appearance of objects, and everything the action does not touch are already visible in the history. The predicted frames must then be turned into a decision, and a generic feature distance to a goal image is a weak measure of task progress. On PushT, ranking the proposals of a diffusion policy by the DINOv2 distance between their true future frames and goal images raises success by only 3 points, whereas a reader trained to rank true futures by task progress raises it by 8.5 points (Tab. 2). A per-step world model that plans with feature distance inherits the first ceiling even if its predictions are perfect.

The opposite extreme predicts nothing. A direct scorer maps the history, a candidate action, and the goal to a score [23, 30]. It is cheap and trained for the decision, but its internal representation is shaped only by the training labels. Nothing asks it to model what the action does to the scene, so whatever it learns about dynamics is entangled with the goals and labels it was trained on.

We argue that a world model for decision making should predict something in between: a compact description of what the action will do that contains only what the decision needs and only what the history does not already reveal. The second requirement is the principle of conditional video coding [26]: when the encoder and the decoder share the past frames, bits need to be spent only on what the past frames do not predict. We apply this principle to the future of an

036
037
038
039
040
041
042
043
044
045
046
047
048
049
050
051
052
053
054
055
056
057
058
059
060
061
062
063
064
065
066
067
068
069
070
071
072
073
074
075

action that has not yet been executed, and we learn the first requirement from task-progress labels instead of fixing it in advance.

We call the resulting model *Conditional Trajectory Abstraction* (CTA, Fig. 1). During training, a target encoder compresses the actual future of an action chunk, including its intermediate frames, into $M=16$ discrete tokens, conditioned on the history. Two heads read this code together with the history but never see the action. A reader ranks sibling candidates by progress toward a goal image, and a feature decoder reconstructs the future’s frozen image features. Because neither head sees the action, everything the action changes must pass through the code, and the decoder prevents the code from degenerating into a score for the training goals. The world model is the predictor of a joint-embedding predictive architecture [2, 25]: it predicts the stop-gradient code from the history and a candidate chunk in one forward pass and never generates an image. At deployment, a frozen policy proposes candidate chunks, the world model predicts one code per candidate in a single batched pass, and the reader ranks the candidates. Since the code does not depend on the goal, a new goal costs only reader evaluations: *predict once, query many*.

Each ingredient of CTA has precedents in world models, video compression, and policy steering, so we state our claims as controlled comparisons. All scorers rank the same proposals, see the same history, and are trained on the same data and labels; they differ in what they predict about the future: every frame, the final frame, nothing, or the conditional code. Our contributions are:

- a formulation of a world model’s prediction target as a rate-limited code of an action’s future segment, conditioned on the history, which contains the per-step latent world model and the direct scorer as its two limiting cases;
- CTA, a joint-embedding world model whose target is learned with a goal-conditioned ranking reader and a conditional feature decoder, and which predicts the code of every candidate in one pass;
- an attribution ladder that scores the same decisions from the true future, from its code, and from the predicted code, separating the ranking quality lost to compression from that lost to prediction;
- experiments on PushT with a released diffusion policy, in which CTA [result vs. policy only, direct scorer, endpoint and per-step latent world models, with planning time].
[TODO: Second task whose success depends on the path.]

2. Related Work

Latent and joint-embedding world models. Latent world models learn dynamics in a compact state space and plan or learn behaviors in imagination [17–19], and token-based

models predict discrete image tokens autoregressively [28]. Δ -IRIS encodes only the stochastic change between consecutive frames given the past [29], the closest per-step relative of conditional coding. Joint-embedding predictive architectures (JEPAs) predict the representation of a target from that of its context instead of reconstructing the target [2, 5, 25]. For control, DINO-WM [38] and V-JEPA 2-AC [3] predict the patch features of a frozen encoder [31] step by step and plan by feature distance to a goal image. CompACT [22] tokenizes each observation into a few discrete tokens to speed up such planning, and GCQ [32] quantizes observation–action sequences with an action-conditioned codebook. In all of these models, the prediction target is a representation of observations, fixed by a pretrained encoder or learned for reconstruction, independently of the decision it will serve. CTA keeps the JEPA recipe of predicting a target representation in latent space, but learns the target itself: one code per action segment, shaped by a decision reader, and conditioned on history that the reader also sees.

Temporally extended prediction. Option models [35] and trajectory-level dynamics models [10, 24] predict the outcome of an extended action without stepping through every intermediate state, and jumpy world models compose such predictions across timescales [13]. Successor features and their action-sequence variants predict cumulative features [6, 7], and TD-Flow models the discounted occupancy of future states [12]. These models summarize the future as a state at one time or as an order-free aggregate, and predict it without reference to what the history already determines. TAP [21] learns discrete codes of trajectory segments conditioned on the current state and searches over them at planning time; its codes include the actions and play the role of latent actions. The code of CTA instead describes the observed consequences of a *given* action chunk, including its intermediate frames, is predicted from that chunk, and spends its capacity only on what the history leaves open.

Policy steering with world models. A frozen generative policy can be improved at deployment by sampling proposals and selecting one with an external scorer. GPC [33], the system closest to ours, ranks diffusion-policy proposals with a learned reward applied to futures predicted by an action-conditioned world model. FOREWARN [36], DreamSteer [9], and DynaGuide [11] steer or rank proposals with vision–language, value, or dynamics models operating on predicted latents, whereas V-GPS [30] and RoboMonkey [23] rank proposals without a world model. We adopt the same propose-then-select interface and, like GPC and V-GPS, train the scorer with task-progress labels. What differs is the world model’s prediction: a goal-independent code of the whole segment that one reader evaluates for any goal image. Our endpoint and per-step latent baselines follow the GPC recipe, but in latent space.

Conditional coding and decision-aware compression.

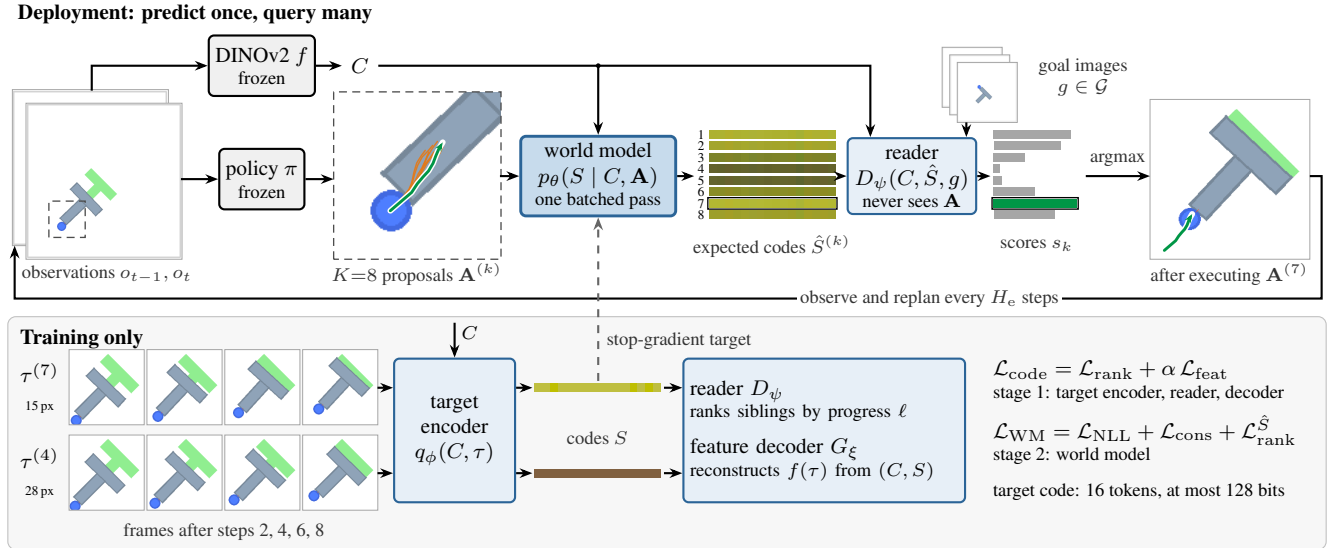


Figure 1. **CTA on a real PushT decision.** *Top:* the context C holds frozen DINOv2 features of the last two observations. The frozen diffusion policy proposes $K=8$ action chunks (orange: the agent’s target positions over 8 steps, shown in the dashed zoomed region). In one batched pass, the world model predicts a distribution over the 16-token code of each chunk’s future segment; each row shows the expected code $\hat{S}^{(k)}$, one cell per token with the token’s three FSQ coordinates as RGB. The reader scores each code against the goal images without seeing the action, and the highest-scoring chunk (green) is executed; a new goal requires only new reader evaluations. *Bottom:* during training, every proposal is executed from a copy of the simulator state. The actual futures of proposals 7 and 4 leave the block’s vertices 15 and 28 pixels from the target pose on average. The target encoder compresses each future into a code S given C ; the reader ranks sibling codes by progress, and the decoder reconstructs the future’s frozen features from (C, S) . The world model is trained to predict the stop-gradient code, and through the frozen reader. Frames are native 512×512 simulator renders, from which the 96×96 observations are downsampled, cropped for visibility; codes and scores are our model’s outputs for this decision.

Learned codecs optimize a rate–distortion trade-off end to end [4], and DCVC [26] conditions both the encoder and the decoder on the temporal context. Unlike the Wyner–Ziv setting [37], in which only the decoder has side information, both our encoder and our heads see the context. Value-equivalent models [16, 34] keep only what is needed to predict values, and Arumugam and Van Roy [1] cast the choice of what to model as a rate–distortion problem. CTA differs from both lines in two ways. Its source is the unexecuted future of an action, which must be predicted before it happens. Its distortion combines a decision term with a generic reconstruction term, so that the code is not specialized to the training goals.

3. Problem Formulation

Setting. At decision time t , a frozen image encoder f maps the robot’s camera images to patch features. The context C collects the features of the current and the previous image together with recent proprioception; it is computed once per decision. A candidate action chunk $A = (a_t, \dots, a_{t+H_e-1})$, if executed, would produce a *future segment* τ : the features of the images it generates, subsampled in time, together with the final proprioception. The world model’s prediction about

τ must suffice to rank the candidates before any of them is executed.

Selecting among policy proposals. At each decision, a frozen pretrained policy π proposes K chunks, where $A^{(1)}$ is the chunk the policy would execute on its own. A scorer ranks the proposals, the highest-ranked chunk is executed, with ties resolved in favor of $A^{(1)}$, and the policy replans after H_e steps [9, 33]. Optimizing actions against a learned model tends to exploit its errors [15]; selecting among the proposals of a competent policy instead keeps the candidates close to the data on which the model was trained [20]. All scorers in this paper rank the same proposals.

Goals and labels. Goals are specified by images $g \in \mathcal{G}$. For training, the simulator labels each candidate with the task progress ℓ reached at the end of its segment; on PushT, ℓ is the negative mean distance between the block’s vertices and their target positions, which orders candidates in the same way as the vertex-registration reward of GPC [33]. Candidates of one decision share their starting state, so only their order matters. No label is used at test time.

Conditional trajectory code. A target encoder q_ϕ , used only during training, maps the context and the actual future to M discrete tokens, $S = q_\phi(C, \tau) \in \{1, \dots, V\}^M$. Two heads read the code together with the context, but never the

	Predicted per candidate	Sequential passes	Supervised by future
Per-step latent WM	$H_e P$ feature tokens	H_e	✓
Endpoint latent WM	P feature tokens	1	✓
Direct scorer	nothing ($S = \mathbf{A}$)	1	—
CTA (ours)	M code tokens	1	✓

Table 1. **What each design predicts** for one candidate chunk, with $P=256$ patch tokens per image, $M=16$ code tokens, and $H_e=8$ on PushT. Except for the direct scorer, the prediction does not depend on the goal and is reused for every goal; only the readout is repeated.

action: a reader $D_\psi(C, S, g)$ that scores progress toward g , and a feature decoder $G_\xi(C, S)$ that reconstructs τ . Given K sibling futures $\tau_1, \dots, \tau_K$ of one decision with labels $\ell_1, \dots, \ell_K$ and codes $S_k = q_\phi(C, \tau_k)$, the code solves the conditional compression problem

$$\begin{aligned} \min_{\phi, \psi, \xi} \mathbb{E} \left[\mathcal{L}_{\text{rank}}(\{D_\psi(C, S_k, g)\}_k, \{\ell_k\}_k) \right. \\ \left. + \frac{\alpha}{K} \sum_k d_{\text{feat}}(G_\xi(C, S_k), \tau_k) \right] \quad (1) \\ \text{s.t. } I(\tau; S | C) \leq R, \end{aligned}$$

where the expectation is over decisions and goals, $\mathcal{L}_{\text{rank}}$ is a ranking loss over the siblings (Sec. 4.3), and d_{feat} is a reconstruction error. Both the encoder and the heads see the context, so the code need not repeat what C already contains. Since S is a deterministic function of (C, τ) , $I(\tau; S | C) = H(S | C) \leq M \log_2 V$; we set the rate through the code length, and $M=16$ tokens with $V=256$ values bound it at 128 bits per segment.

A world model $p_\theta(S | C, \mathbf{A})$ must forecast the code before execution. Given the context, the action can reduce the uncertainty about the code by at most $I(S; \mathbf{A} | C)$; the remainder, $H(S | C, \mathbf{A})$, reflects dynamics that the context and the action do not determine and cannot be forecast. We estimate $I(S; \mathbf{A} | C)$ in Sec. 4.4.

Two limiting cases. The family in (1) contains two familiar designs as extremes (Tab. 1). A *per-step latent world model* such as DINO-WM sets $S = \tau$: it predicts every token of every future frame, including everything the history already determines, and ranks with a fixed feature distance or a learned readout. A *direct scorer* $D(C, \mathbf{A}, g)$ sets $S = \mathbf{A}$: nothing about the future is predicted, and the reader must learn the dynamics together with the evaluation. CTA places S between these extremes, as a learned description of the future at a bounded rate that the reader can evaluate for any goal.

4. Conditional Trajectory Abstraction

CTA has three learned components: a target encoder that turns the actual future into a code (Sec. 4.2), two heads that read the code (Sec. 4.3), and a world model that predicts the code from an action chunk (Sec. 4.4). We describe them, the training procedure (Sec. 4.5), and deployment (Sec. 4.6) in turn.

4.1. Inputs

As in DINO-WM [38], the image encoder f is a frozen DINOv2 ViT-S/14 [31]; we project its patch features to 128 dimensions with PCA fitted on training frames. The context C contains the 16×16 patch tokens of the current image, the previous image pooled to 8×8 tokens, and the proprioception at both times. The future segment τ contains the full token grid of the final image, the pooled tokens of three intermediate images with temporal embeddings, and the final proprioception. Features are computed once per image and shared by all candidates and goals.

4.2. Target encoder

The target encoder $q_\phi(C, \tau)$ is used only during training. M learned queries cross-attend to the tokens of the future and of the context, and each output is quantized with FSQ [27] using levels (8, 8, 4), so that each token takes one of $V=256$ values. Because the queries also attend to the context, the encoder can learn which parts of the future the history already implies and spend the code on the rest. The M tokens describe the segment as a whole rather than one frame each.

4.3. Reader and feature decoder

The reader $D_\psi(C, S, g)$ is a Transformer over the context tokens, the code tokens, and the patch tokens of a goal image, and it outputs a single score. It is trained on sibling candidates that branch from the same state, with a pairwise logistic loss over the sibling pairs whose labels differ by more than a margin ϵ :

$$\mathcal{L}_{\text{rank}} = \mathbb{E}_{(i,j): \ell_i > \ell_j + \epsilon} \left[w_{ij} \log(1 + e^{-(s_i - s_j)}) \right], \quad (2)$$

where $s_i = D_\psi(C, S_i, g)$ and $w_{ij}=1$ when training the reader. When the loss is applied to predicted codes (Sec. 4.4), $w_{ij} = \min(1, (\ell_i - \ell_j)/\sigma)$ down-weights pairs whose labels barely differ. Ranking within a decision matches how the score is used, so the score does not need to be calibrated across states.

A ranking loss alone would let the code shrink to whatever orders the candidates for the training goals. The feature decoder $G_\xi(C, S)$ prevents this by reconstructing the future tokens from the context and the code. It predicts the change of each token relative to the current image, and its squared-error loss $\mathcal{L}_{\text{feat}}$ is normalized by the error of predicting no change, so that a code that carries no information scores

one. The decoder thus plays the role that EMA targets and variance regularizers play in other JEPAs [2, 25]: it keeps the target from degenerating, here into a score for the training goals. No image is ever generated.

4.4. World model

The world model $p_\theta(S | C, \mathbf{A})$ is the predictor of the JEPA. A Transformer encoder reads the context tokens and one token per action, and M learned queries decode all code tokens in parallel. Each FSQ coordinate of each token is predicted as a categorical distribution over its levels, so one forward pass yields a distribution over codes for every candidate in the batch. The reader receives the expected codeword $\hat{S} = \mathbb{E}_{p_\theta}[S]$, which exposes the world model’s uncertainty to the reader and avoids the ties that the most likely code produces among similar candidates (Sec. 5.4). The 128-bit bound therefore applies to the target code; the deployed input to the reader is the predictive mean, $M \times 3$ numbers in the FSQ cube. The world model never sees future images, the goal, or labels.

The target encoder and the reader are frozen while the world model is trained against the stop-gradient code $S = \text{sg}(q_\phi(C, \tau))$:

$$\mathcal{L}_{\text{WM}} = \mathcal{L}_{\text{NLL}} + \mathcal{L}_{\text{cons}} + \mathcal{L}_{\text{rank}}^{\hat{S}}. \quad (3)$$

$\mathcal{L}_{\text{NLL}}$ is the cross-entropy of each coordinate of S . The other two terms pass gradients through the frozen reader into the world model. $\mathcal{L}_{\text{cons}}$ matches the within-decision differences between the reader’s scores of $\hat{S}$ and of S , and $\mathcal{L}_{\text{rank}}^{\hat{S}}$ applies the weighted form of (2) to the scores of the predicted codes. A context-only model $r_\omega(S | C)$ is trained with $\mathcal{L}_{\text{NLL}}$ alongside the world model. Its excess cross-entropy over the world model estimates $I(S; \mathbf{A} | C)$, the part of the code that the action explains.

4.5. Training

Branched data. We roll out the frozen policy and, at every decision, execute each of its K proposals for H_e steps from an exact copy of the simulator state. This yields K sibling futures and labels that share one history. The proposals of a competent policy often lead to nearly identical futures, so we also branch a perturbed copy of each proposal, shifted by a constant planar offset whose scale grows across the copies. To cover the states that a selector visits, not only those that the policy visits, some rollouts execute the best-labeled proposal at a random half of their decisions. All branches of a rollout stay in the same data split.

Stages. Stage 1 trains the target encoder, the reader, and the feature decoder on actual futures with $\mathcal{L}_{\text{code}} = \mathcal{L}_{\text{rank}} + \alpha \mathcal{L}_{\text{feat}}$, using straight-through gradients through the quantizer. Stage 2 freezes them and trains the world model with (3). Each network keeps the checkpoint with the

Algorithm 1 Selecting among policy proposals with CTA.

Require: frozen policy π and image encoder f ; world model p_θ ; reader D_ψ ; goal images $\mathcal{G}$

- 1: **while** episode not finished **do**
- 2: $C \leftarrow$ context from f applied to the history ▷ once per decision
- 3: $\{\mathbf{A}^{(k)}\}_{k=1}^K \leftarrow$ proposals of π
- 4: $\hat{S}^{(k)} \leftarrow \mathbb{E}_{p_\theta(\cdot | C, \mathbf{A}^{(k)})}[S]$ for all k ▷ one batched pass
- 5: $s_k \leftarrow \frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} D_\psi(C, \hat{S}^{(k)}, g)$
- 6: $k^* \leftarrow$ smallest index attaining $\max_k s_k$ ▷ ties keep $\mathbf{A}^{(1)}$
- 7: execute $\mathbf{A}^{(k^*)}$; $t \leftarrow t + H_e$

best value of its own objective on held-out development rollouts. Throughout training we monitor the code perplexity, the number of distinct codes per decision, the decoder’s R^2 , and $I(S; \mathbf{A} | C)$ to detect collapse.

Co-design (ablation). The code can also be shaped to be predictable. An optional third stage alternates blocks of target-encoder and world-model updates and adds $\lambda \mathbb{E}[-\log \bar{p}_\theta(S | C, \mathbf{A})]$ to $\mathcal{L}_{\text{code}}$, where $\bar{p}_\theta$ is a frozen copy of the world model and the discrete likelihood is interpolated multilinearly on the FSQ grid to obtain a gradient. We report it as an ablation (Sec. 5.4).

4.6. Planning

Algorithm 1 summarizes deployment. At each decision, CTA encodes the images once, runs the world model once on all proposals, and runs the reader on every candidate–goal pair. A new goal therefore costs only reader evaluations.

4.7. Implementation details

All modules have width 256 and 8 attention heads. The target encoder has three Transformer decoder layers, the reader four encoder layers, the feature decoder two decoder layers, and the world model four encoder and four decoder layers. We use $M=16$ (128 bits per target code), $\alpha=1$, $\epsilon=10^{-3}$, and $\sigma=10^{-2}$. A small penalty on FSQ pre-activations beyond ± 1.5 keeps the quantizer out of saturation, where straight-through gradients vanish. Stage 2 runs 8,000 AdamW updates with learning rate 10^{-4} , weight decay 0.05, and bfloat16 autocast, with 32 decisions per update. **[TODO: Final stage-1 schedule, trained from scratch; current checkpoints are warm-started across development rounds.]** On PushT, the 96×96 images are resized to 224×224 for DINOv2, $H_e=8$, and the intermediate images follow steps 2, 4, and 6 of the chunk. Each action is a target position of the agent and is embedded in absolute coordinates and relative to the agent’s current position.

5. Experiments

We use CTA to select among the proposals of a frozen diffusion policy and ask three questions. Does CTA improve

the policy, and how does it compare in success and planning time with latent world models and a direct scorer that rank the same proposals (Sec. 5.2)? Where between the true future and the predicted code is ranking quality lost (Sec. 5.3)? Which design choices matter (Sec. 5.4)? Section 5.5 turns to a task whose success depends on what happens within a segment.

5.1. Setup

Task and policy. On PushT [14], an episode succeeds when the block covers more than 95% of the target region within 300 steps. We use the released LeRobot Diffusion Policy [8], which conditions on two observations, denoises 16 actions, and executes 8 of them. Unmodified, it succeeds in 65 of 100 of our qualification episodes, against 65.4% reported on 500 episodes. At each decision, it proposes $K=8$ chunks from independent, pre-generated denoising noise, and the policy-only baseline executes $\mathbf{A}^{(1)}$ at every decision. **[TODO: Lock the deployment bank: the $K=8$ proposals only, or also their perturbed copies.]**

Data. Training uses 2,000 branched rollouts (Sec. 4.5) with 55,270 decisions. In 1,200 of these rollouts each decision is also branched with perturbed copies of the proposals, and half of them execute the best-labeled proposal at a random half of their decisions; this gives 87,860 banks of eight sibling segments. A further 100 rollouts are used for offline model selection and the attribution ladder, 200 episodes for closed-loop development, and a sealed set of 400 episodes, evaluated once, for the results we report, with three training seeds. PushT has a single target pose, so the 16 goal images, the final frames of successful episodes outside all splits, differ only in the position of the agent and small deviations of the block. PushT therefore does not test the reuse of predictions across different goals.

Baselines. All learned scorers use the same frozen DINOv2 features, context, proposals, training data, labels, ranking loss, and checkpoint selection. The *direct scorer* $D(C, \mathbf{A}, g)$ is a Transformer over the context, action, and goal tokens, with eight layers so that its capacity is comparable to that of the world model and reader combined. The *factorized direct scorer* maps $(C, \mathbf{A})$ to a vector of the same size as $\hat{S}$ and reads it with the reader architecture; it is trained with the ranking loss only, and isolates the effect of supervising this bottleneck with the future. The *endpoint latent world model* predicts the features of the final image and the final proprioception in one pass and is read by a reader trained on true futures. The *per-step latent world model* predicts the features of every frame autoregressively, in the manner of DINO-WM [38], and is read in the same way. **[TODO: Implement the factorized direct scorer and the per-step latent world model.]** We also report three privileged references that simulate every candidate and are therefore not deployable: the *oracle*, which picks the candidate with the

	Success (%)	Score	ms / decision	
			Scoring	Total
Policy only	—	—	—	—
Direct scorer	—	—	—	—
Factorized direct scorer	—	—	—	—
Endpoint latent WM	—	—	—	—
Per-step latent WM	—	—	—	—
CTA (ours)	—	—	—	—
<i>Privileged: simulates every candidate</i>				
Feature distance, true future	—	—	—	—
Reader, true future	—	—	—	—
Oracle	—	—	—	—

Table 2. **Closed-loop results on PushT** (sealed set of 400 episodes, mean over three training seeds). All methods select among the same $K=8$ proposals. *Scoring* is the time of the world model and reader for all candidates; *Total* adds image encoding and proposal sampling.

best label; the *reader on the true future*, which reads the true final image and proprioception; and *feature distance on the true future*, which is DINO-WM’s planning cost under perfect prediction.

Metrics. We report closed-loop success and the mean episode score, defined as the maximum coverage reached in an episode divided by the 95% success threshold and clipped at one. Intervals are paired bootstrap intervals over episodes, and success differences are also tested with exact McNemar tests. **[TODO: Fix the primary metric before the sealed evaluation (the development protocol uses the score).]** For planning time we report the scoring time, meaning the world model and reader for all K candidates, and the total time per decision including image encoding and proposal sampling, on the same GPU. Offline, the *retained gap* of a scorer is the fraction of the oracle’s gain over the default that its choices recover,

$$\text{RG} = \frac{\sum_d (\ell_d(\hat{k}_d) - \ell_d(1))}{\sum_d (\max_k \ell_d(k) - \ell_d(1))}, \quad (4)$$

where d ranges over decisions, $\hat{k}_d$ is the scorer’s choice, and $\ell_d(1)$ is the label of $\mathbf{A}^{(1)}$.

5.2. Main results

[Main result, in this order: CTA vs. policy only; vs. the direct and factorized direct scorers; vs. the endpoint and per-step latent world models at their measured cost; fraction of the oracle’s and the true-future reader’s gain recovered.]

5.3. Where is ranking quality lost?

Table 3 scores the same decisions while the reader’s input moves from the true future to its code and then to the code

Reader input	Retained gap
True future (final image and proprioception)	—
Code of the true future, S	—
Predicted code, $\hat{S}$ (CTA)	—
Predicted code, NLL-only world model	—
Predicted final-image features (endpoint WM)	—
Actions only (direct scorer)	—

Table 3. **Attribution ladder** on the development decisions, with 95% intervals clustered by episode. The drop from the second to the third row is lost to prediction. The first row reads a different view of the true future than the code does, so it is a reference rather than an upper bound.

	RG	Success (%)
CTA	—	—
Target encoder without history	—	—
Target from the final frame only	—	—
Per-step code, same total bits	—	—
No feature decoder ($\alpha=0$)	—	—
World model with $\mathcal{L}_{\text{NLL}}$ only	—	—
Most likely code instead of $\hat{S}$	—	—
Co-design, $\lambda>0$	—	—
$M=8 / M=32$	—	—

Table 4. **Ablations.** Each row changes one choice and keeps the data and training budget fixed. RG: offline retained gap on development decisions; success on the closed-loop development episodes.

predicted from the action. [Interpretation, with code diagnostics: perplexity, distinct codes per decision, distinct token values per code, $I(S; A | C)$ in bits, feature R^2 .]

5.4. Ablations

Each ablation in Tab. 4 changes one choice. Removing the history from the target encoder tests conditional coding. Restricting the target to the final frame, or replacing the segment code with one code per frame at the same total number of bits, tests the segment design; if either matched CTA, the segment design would not be supported. The remaining rows test the feature decoder, the decision terms of the world-model loss, deployment with the expected code, co-design for predictability, and the code length. [Findings.]

5.5. When progress depends on the path

[TODO: Second task whose success depends on what happens within a segment. Key contrasts: CTA vs. the final-frame code at equal bits, and vs. the per-step latent world model at equal quality.]

6. Conclusion and Limitations

We proposed to make the prediction target of a world model a learned, history-conditioned code of an action’s entire future segment, rather than one latent state per time step. CTA learns this code with a decision reader and a feature decoder, predicts it in one pass, and lets one prediction per candidate be read for any goal. [What the experiments show, and what they do not.]

Limitations. CTA is not label-free: task-progress labels from the simulator train the reader and the decision terms of the world-model loss, although nothing uses labels at test time. Training data require branching the simulator at every decision. The proposal policy is pretrained on expert demonstrations, and a selector cannot do better than the best proposal, so gains are bounded by the policy’s proposals. The rate bound applies to the target code; at deployment the reader receives the predictive mean of the code. Finally, PushT has a single goal, so it tests neither the reuse of predictions across goals nor the value of the intermediate part of the segment; Sec. 5.5 addresses the latter.

References

- [1] Dilip Arumugam and Benjamin Van Roy. Deciding what to model: Value-equivalent sampling for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 3
- [2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2, 5
- [3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 1, 2
- [4] Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. In *International Conference on Learning Representations (ICLR)*, 2017. 3
- [5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research (TMLR)*, 2024. 2
- [6] André Barreto, Will Dabney, Rémi Munos, Jonathan J. Hunt, Tom Schaul, Hado van Hasselt, and David Silver. Successor features for transfer in reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2
- [7] Boyuan Chen, Chuning Zhu, Pulkit Agrawal, Kaiqing Zhang, and Abhishek Gupta. Self-supervised reinforcement learning that transfers using random features. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2
- [8] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffu-

- sion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023. 6
- [9] Hanchen Cui et al. DreamSteer: Latent world models can steer VLA policies during deployment without any finetuning. *arXiv preprint arXiv:2607.02865*, 2026. 2, 3
- [10] Zihan Ding, Amy Zhang, Yuandong Tian, and Qingqing Zheng. Diffusion world model: Future modeling beyond step-by-step rollout for offline reinforcement learning. *arXiv preprint arXiv:2402.03570*, 2024. 2
- [11] Maximilian Du and Shuran Song. DynaGuide: Steering diffusion policies with active dynamic guidance. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. 2
- [12] Jesse Farebrother, Matteo Pirota, Andrea Tirinzoni, Rémi Munos, Alessandro Lazaric, and Ahmed Touati. Temporal difference flows. In *International Conference on Machine Learning (ICML)*, 2025. 2
- [13] Jesse Farebrother, Matteo Pirota, Andrea Tirinzoni, Marc G. Bellemare, Alessandro Lazaric, and Ahmed Touati. Compositional planning with jumpy world models. *arXiv preprint arXiv:2602.19634*, 2026. 2
- [14] Pete Florence, Corey Lynch, Andy Zeng, Oscar Ramirez, Ayzaan Wahid, Laura Downs, Adrian Wong, Johnny Lee, Igor Mordatch, and Jonathan Tompson. Implicit behavioral cloning. In *Conference on Robot Learning (CoRL)*, 2021. 6
- [15] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023. 3
- [16] Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 3
- [17] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*, 2019. 1, 2
- [18] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025. 1
- [19] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024. 2
- [20] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. 3
- [21] Zhengyao Jiang, Tianjun Zhang, Michael Janner, Yueying Li, Tim Rocktäschel, Edward Grefenstette, and Yuandong Tian. Efficient planning in a compact latent action space. In *International Conference on Learning Representations (ICLR)*, 2023. 2
- [22] Dongwon Kim, Gawon Seo, Jinsung Lee, Minsu Cho, and Suha Kwak. Planning in 8 tokens: A compact discrete tokenizer for latent world model. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. 2
- [23] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. RoboMonkey: Scaling test-time sampling and verification for vision-language-action models. In *Conference on Robot Learning (CoRL)*, 2025. 1, 2
- [24] Nathan O. Lambert, Albert Wilcox, Howard Zhang, Kristofer S. J. Pister, and Roberto Calandra. Learning accurate long-term dynamics for model-based reinforcement learning. In *IEEE Conference on Decision and Control (CDC)*, 2021. 2
- [25] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, version 0.9.2, 2022. 2, 5
- [26] Jiahao Li, Bin Li, and Yan Lu. Deep contextual video compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 1, 3
- [27] Fabian Mentzer, David Minnen, Eiríkur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations (ICLR)*, 2024. 4
- [28] Vincent Micheli, Elói Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023. 2
- [29] Vincent Micheli, Elói Alonso, and François Fleuret. Efficient world models with context-aware tokenization. In *International Conference on Machine Learning (ICML)*, 2024. 2
- [30] Mitsuhiro Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. In *Conference on Robot Learning (CoRL)*, 2024. 1, 2
- [31] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024. 2, 4
- [32] Xiangyuan Peng, Kingsi Dong, and Si Wu. Vector quantization in the brain: Grid-like codes in world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. 2
- [33] Han Qi, Haocheng Yin, Aris Zhu, Yilun Du, and Heng Yang. Inference-time enhancement of generative robot policies via predictive world modeling. *IEEE Robotics and Automation Letters (RA-L)*, 2025. 2, 3
- [34] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020. 3
- [35] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. 2
- [36] Yilin Wu, Ran Tian, Gokul Swamy, and Andrea Bajcsy. From foresight to forethought: VLM-in-the-loop policy steering via latent alignment. In *Robotics: Science and Systems (RSS)*, 2025. 2
- [37] Aaron D. Wyner and Jacob Ziv. The rate-distortion function for source coding with side information at the decoder. *IEEE Transactions on Information Theory*, 22(1):1–10, 1976. 3

- 656 [38] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto.
657 DINO-WM: World models on pre-trained visual features en-
658 able zero-shot planning. In *International Conference on Ma-*
659 *chine Learning (ICML)*, 2025. [1](#), [2](#), [4](#), [6](#)